

Linux Kernel & OS Tuning for PostgreSQL: An Expert Practitioner's Reference

TL;DR

- The highest-impact OS settings for a dedicated PostgreSQL server are: disable Transparent Huge Pages (THP) while enabling **explicit** huge pages for `(shared_buffers)`; set `(vm.swappiness=1)`; use absolute dirty-page limits `(vm.dirty_bytes)`/`(vm.dirty_background_bytes)` rather than the default percentages; set `(vm.zone_reclaim_mode=0)` and `(kernel.numa_balancing=0)` on NUMA hardware; use the `(none)` I/O scheduler on NVMe; pin the CPU governor to `(performance)`; and protect the postmaster from the OOM killer `(OOMScoreAdjust=-1000)` plus `(vm.overcommit_memory=2)`.
- Since PostgreSQL 9.3 moved to POSIX/`(mmap)` shared memory, `(kernel.shmmax)`/`(shmall)` are largely irrelevant for startup, but semaphores `(kernel.sem)`, file descriptors `(LimitNOFILE)`, locked memory `(LimitMEMLOCK)` for huge pages, and TCP keepalives remain critical — and PostgreSQL 18's new asynchronous I/O `(io_method=worker)`/`(io_uring)` adds a new kernel dependency `(kernel.io_uring_disabled=0)`.
- Tuning must be workload-aware: OLTP favors low, predictable write latency (tight dirty limits, aggressive checkpoint spreading); OLAP/analytics favors read throughput and larger read-ahead / higher `(effective_io_concurrency)`; high-connection and replication deployments need larger `(smaxconn)`, backlog, and keepalive tuning — plus connection pooling rather than raw `(max_connections)`.

Key Findings

1. PostgreSQL deliberately co-operates with the kernel. Unlike Oracle or (historically) MySQL/InnoDB with `O_DIRECT`, PostgreSQL relies on the Linux page cache for buffered reads and writes of data files, and on the kernel's writeback machinery to destage dirty pages. This is still true by default through PostgreSQL 18 — Direct I/O exists only as a debug option, and the new AIO subsystem covers reads, not writes or WAL. Consequently, kernel VM and writeback settings directly shape checkpoint behavior and latency.

2. Memory overcommit and the OOM killer are the single biggest reliability risk. The default `(vm.overcommit_memory=0)` (heuristic overcommit) lets the kernel kill the postmaster under memory pressure, forcing a crash-recovery cycle and dropping all connections. The PostgreSQL docs recommend `(vm.overcommit_memory=2)` and/or protecting the postmaster with `(oom_score_adj=-1000)`.

3. Huge pages help; THP hurts. Explicit huge pages reduce TLB misses and page-table overhead for large `(shared_buffers)` and are never swapped; THP's background compaction/`(khugepaged)` causes latency spikes and should be disabled or set to `(madvise)`. In Percona's "Settling the Myth of Transparent HugePages for Databases," `pgbench` TPCB and `sysbench` TPCC tests at 48 GB and 112 GB database sizes with 64/128/256 clients showed that "both lines — representing execution with or without THP — are almost overlapping one another. This suggests no performance gains." (Percona)

4. On modern NVMe, the kernel I/O scheduler should get out of the way `(none)`, while spinning disks and SATA SSDs generally benefit from `(mq-deadline)`. Red Hat's RHEL 9 documentation states plainly: "For non-volatile Memory Express (NVMe) block devices specifically, the default scheduler is `(none)` and Red Hat recommends not changing this." (Red Hat)

5. PostgreSQL 18 changes the I/O tuning landscape with asynchronous I/O `(io_method)`, making `(effective_io_concurrency)` an active control (its default was raised from 1 to 16) and introducing NUMA observability views.

Details

1. Memory Management `(vm.*)`

`vm.overcommit_memory` / `vm.overcommit_ratio`

- Where:** `(/etc/sysctl.conf)` or `(/etc/sysctl.d/*.conf)`; runtime via `(sysctl -w)`.
- Controls:** Kernel policy for allowing allocations beyond physical RAM+swap. `(0)`=heuristic (default), `(1)`=always overcommit, `(2)`=strict/no-overcommit beyond `(CommitLimit = swap + RAM*overcommit_ratio/100)`.
- Default:** `(overcommit_memory=0)`, `(overcommit_ratio=50)`.
- Recommended:** `(vm.overcommit_memory=2)`; raise `(vm.overcommit_ratio)` (commonly ~80) on RAM-heavy DB servers, or use `(vm.overcommit_kbytes)` for absolute control.
- Why (internals):** With strict overcommit, when a backend's `(malloc)` cannot be satisfied the kernel returns `ENOMEM` and that backend errors out gracefully; the postmaster and other backends survive. Under the default policy the OOM killer may instead `SIGKILL` the postmaster, which assumes shared-memory corruption and forces a full crash-recovery / connection reset.
- Trade-offs / caveats:** On swapless systems `(overcommit_memory=2)` can cause allocations to fail earlier than expected; size `(CommitLimit)` to cover `(shared_buffers + max_connections*work_mem*(operators) + autovacuum workers*maintenance_work_mem)`. Inside cgroup

v2/Kubernetes, `memory.max` triggers an immediate SIGKILL with no ENOMEM grace, so `overcommit=2` does not help there — size the cgroup instead.

vm.swappiness

- **Where:** `sysctl`. **Controls:** kernel's propensity (0–100) to swap anonymous pages vs. evict page cache. **Default:** 60.
- **Recommended:** `1` (not `0`).
- **Why:** DB servers manage their own memory; swapping backend memory adds unpredictable latency. `0` can raise OOM-kill risk on newer kernels, so `1` is the safe minimum that still allows swap as a last resort. (There is genuine community debate: Cybertec's Laurenz Albe has argued for `0` since PostgreSQL does its own memory management; others argue leaving 60 so idle backends can be swapped out. For predictable OLTP latency, `1` is the mainstream recommendation.)

vm.dirty_ratio / vm.dirty_background_ratio and vm.dirty_bytes / vm.dirty_background_bytes

- **Where:** `sysctl`. **Controls:** thresholds at which the kernel starts background writeback (`*_background_*`) and forces synchronous, blocking writeback (`(dirty_ratio)/(dirty_bytes)`). Ratios are percent of available RAM; the `_bytes` forms are absolute (setting one zeroes the other).
- **Defaults:** `(dirty_background_ratio=10)`, `(dirty_ratio=20)`; `(dirty_expire_centisecs=3000)` (30 s), `(dirty_writeback_centisecs=500)` (5 s).
- **Recommended:** On large-RAM servers, percentage-based limits allow gigabytes of dirty data to accumulate and then flush in a storm during checkpoint `fsync`, causing I/O stalls. Prefer absolute limits, e.g. `(vm.dirty_background_bytes)` sized near the storage/RAID cache and `(vm.dirty_bytes)` a few times larger (e.g. background 64–256 MB, hard limit 256 MB–1 GB depending on device bandwidth). Keep `(dirty_ratio > dirty_background_ratio)` so background writeback starts before the blocking limit.
- **Why (internals):** The checkpointer writes dirty shared buffers to the page cache, then `fsyncs`. If the kernel has hoarded dirty pages, that `fsync` becomes a long stall affecting all backends. Since 9.6, `(checkpoint_flush_after)` (default 256 kB) makes PostgreSQL `fsync` incrementally during checkpoints, greatly reducing the need to tune these kernel knobs — but they still matter for backend writes and non-checkpoint writeback. Coordinate with `(checkpoint_completion_target)` (default 0.9), `(max_wal_size)`, and `bgwriter` settings (`(bgwriter_lru_maxpages)`, `(bgwriter_flush_after)`).
- **Workload:** Write-heavy OLTP → tighter absolute limits for smoother latency; bulk-load/OLAP → can tolerate larger limits for throughput.

vm.dirty_expire_centisecs / vm.dirty_writeback_centisecs

- **Controls:** max age of a dirty page before writeback (3000 cs/30 s default) and how often the flusher threads wake (500 cs/5 s default). Lowering `(dirty_expire_centisecs)` (e.g. to 500) forces more frequent, smaller writebacks — useful pre-9.6 or for very latency-sensitive systems.

vm.zone_reclaim_mode

- **Where:** `sysctl`. **Controls:** whether the kernel reclaims memory within a NUMA zone before allocating from a remote node. **Default:** auto-enabled by the kernel when inter-node distance is large (value 1).
- **Recommended:** `0`.
- **Why:** When enabled, the kernel aggressively reclaims (evicts page cache) on the local node rather than using remote RAM, which for a database starves the page cache and causes swapping/latency despite free remote memory. Greg Smith (2ndQuadrant) recommends always disabling it for PostgreSQL. (PostgreSQL)

vm.min_free_kbytes

- **Controls:** amount of RAM the kernel keeps free as an emergency reserve for atomic allocations. Raising it modestly on large-memory servers (e.g. 1–2 GB) reduces the chance of allocation stalls/latency spikes under memory pressure; too high wastes RAM.

Huge pages: vm.nr_hugepages, vm.hugetlb_shm_group, huge_pages GUC

- **Where:** `(vm.nr_hugepages)` via `sysctl` or GRUB (`(hugepagesz=2M hugepages=N)`); `(vm.hugetlb_shm_group)` via `sysctl`; `(huge_pages)/(huge_page_size)` in `postgresql.conf`.
- **Controls:** number of explicitly reserved huge pages (default page 2 MB; 1 GB available). PostgreSQL uses them for the main shared memory segment (`(shared_buffers)` etc.).
- **Recommended:** Size with `(postgres -D $PGDATA -C shared_memory_size_in_huge_pages)` (a runtime-computed GUC), reserve that many + headroom via `(vm.nr_hugepages)`, set `(huge_pages=on)` (fail-fast) or `(try)` (default). Grant the `postgres` group via `(vm.hugetlb_shm_group)` and allow `(memlock)` (`(LimitMEMLOCK)` / `(ulimit -l)`).

- **Why:** Fewer, larger page-table entries → fewer TLB misses and page faults for tens-of-GB (`shared_buffers`); huge pages are locked in RAM and never swapped. Must disable THP first, or the kernel may back the segment with THP inconsistently.
- **Caveats:** Reserved huge pages are unavailable to other processes; over-reserving wastes RAM. PostgreSQL reads only the first (`Hugepagesize`) line of (`/proc/meminfo`), so mixing 2 MB/1 GB sizes needs (`huge_page_size`). Best on dedicated servers. PG18 also exposes (`huge_pages_status`) to confirm actual use.

Transparent Huge Pages (THP)

- **Where:** (`/sys/kernel/mm/transparent_hugepage/enabled`) and (`/defrag`); persist via GRUB (`transparent_hugepage=never`) or a systemd oneshot unit (ordered (`Before=postgresql.service`)).
- **Recommended:** (`never`) for dedicated DB hosts (or (`madvise`) on mixed-use hosts so other apps like JVMs can opt in). Set defrag to (`never`) (`madvise`) (or (`defer+madvise`) on kernels ≥ 4.6).
- **Why:** khugepaged/direct compaction to assemble 2 MB pages causes latency spikes and high system CPU; database access patterns are sparse, so THP yields little benefit and real harm. As noted above, Percona's benchmarks found overlapping throughput lines with and without THP ("no performance gains"). Disable THP before using explicit huge pages. (`Stormatics`) (`Stormatics`)

vm.max_map_count

- **Controls:** max number of memory-map areas per process (default 65530). Large numbers of partitions, extensions, or a very large shared-memory footprint can approach it; raise (e.g. 262144+) for very large/parallel deployments to avoid (`mmap`) failures.

OOM killer: oom_score_adj, oom_kill_allocating_task

- **Where:** (`/proc/<pid>/oom_score_adj`); systemd (`OOMScoreAdjust`); env (`PG_OOM_ADJUST_FILE`) (`PG_OOM_ADJUST_VALUE`).
- **Recommended:** Protect the postmaster with (`oom_score_adj=-1000`) (systemd (`OOMScoreAdjust=-1000`); Debian/Ubuntu units ship (`-900`)). Set (`PG_OOM_ADJUST_FILE=/proc/self/oom_score_adj`) and (`PG_OOM_ADJUST_VALUE=0`) so child backends reset to a killable score — this ensures a runaway backend, not the postmaster, is sacrificed.
- **Why:** Killing the postmaster or any backend forces the whole cluster into crash recovery (the postmaster assumes shared memory may be corrupt), dropping all connections and possibly losing async-replicated committed transactions. Keep (`oom_kill_allocating_task=0`) (default) so the kernel picks the highest-scoring process rather than whoever happened to allocate. (Crunchy Data's "The Linux Assassin" details why OOM-kill avoidance is so important for HA.)

2. NUMA

- **vm.zone_reclaim_mode=0** (see above) — the most important NUMA setting.
- **kernel.numa_balancing=0:** Automatic NUMA balancing migrates pages toward the CPUs touching them; for a shared-memory database with many backends touching (`shared_buffers`) this causes wasteful page migration and TLB shootdowns. Recommended off for a single dedicated cluster (Cybertec's Laurenz Albe: "kernel.numa_balancing should be 0").
- **numactl --interleave=all:** Historically recommended to spread (`shared_buffers`) evenly across nodes and avoid one node filling up. Modern consensus (Vondra, Freund on pgsql-hackers): interleaving the *whole* postmaster also interleaves per-backend private memory (work_mem, stacks), which can hurt OLAP. The cleaner path is to use explicit huge pages (reserved at boot, spread evenly across nodes, never swapped) and leave (`zone_reclaim_mode=0`), rather than blanket interleaving. Kevin Grittner's tests showed interleaving OS cache + (`shared_buffers`) gave ~2% on read-only loads; conversely, a deliberately misconfigured case (DB forced onto a distant node) dropped throughput to ~20% of the balanced case. (`PostgreSQL`) (`Postgres Professional`)
- **PostgreSQL 18 NUMA awareness:** PG18 added build option (`--with-libnuma`), the function (`pg_numa_available()`), and views (`pg_shmem_allocations_numa`) and (`pg_buffercache_numa`) for observing shared-memory/buffer distribution across nodes. These are **observability only** — the docs warn (`pg_shmem_allocations_numa`) "should not be queried by monitoring systems as it is very slow and may end up allocating shared memory." The actual (`numa`) (`numa_buffers_interleave`) (`numa_localalloc`) GUCs are development patches targeting PostgreSQL 19, not shipped in 18. (`PostgreSQL`)
- **BIOS:** Disabling node interleaving in BIOS vs. leaving NUMA visible to the OS is debated; Andres Freund favors keeping NUMA visible and letting PostgreSQL/huge pages handle placement.

3. Storage / I/O and Filesystem

I/O scheduler

- **Where:** (`/sys/block/<dev>/queue/scheduler`); persist via udev rule (the legacy (`elevator=`) kernel arg is ignored under blk-mq on kernel ≥ 5.0).

- **Recommended:** NVMe → `(none)`; SATA/SAS SSD → `(mq-deadline)` (or `(none)`); HDD → `(mq-deadline)` (or `(bfq)` for interactive fairness); VM guests (virtio) / hardware RAID with write-back cache → `(none)`. Per Red Hat's RHEL 9 guidance: `(none)` "is recommended for systems that have high performance storage like Solid State Drives (SSD) or NVMe drives"; `(mq-deadline)` "is suited for traditional HDD storage"; `(bfq)` "is recommended for desktop or interactive tasks." ([Dell Technologies Info Hub...](#))
- **Why:** NVMe firmware does its own deep multi-queue scheduling; adding software reordering only adds CPU and latency. Scheduler choice mainly affects tail latency (p99/p999), not median. Also tune `(nr_requests)` (NVMe can use deep queues, e.g. 1024–4096) and `(read_ahead_kb)`.

Read-ahead (`blockdev --setra / read_ahead_kb`)

- Default 128 kB (256 sectors). For sequential/OLAP scans and backups, increase (e.g. `(blockdev --setra 4096)-(16384)`, i.e. 2–8 MB) to improve throughput; for pure random OLTP it matters less. In PG18, ReadStream + AIO reduces reliance on kernel read-ahead for seq scans, bitmap scans, and VACUUM (but not plain index scans, which still benefit from OS read-ahead when index and heap order roughly match).

Filesystem choice and mount options

- **Recommended:** XFS or ext4 are the mainstream, well-tested choices (XFS is often preferred for large PostgreSQL clusters). ZFS offers checksums/snapshots but adds double-buffering considerations (tune ARC, set `(recordsize)` appropriately, and consider `(full_page_writes=off)` only with ZFS's atomic-write guarantee).
- **Mount options:** `(noatime)` (avoid metadata writes on read) is standard. For ext4, `(data=ordered)` (default) is safe; `(data=writeback)` is faster but riskier. Do **not** disable barriers (`(nobARRIER)`) unless you have a battery-/flash-backed write cache (BBU) that makes the volatile-cache assumption false — otherwise you risk corruption on power loss.
- **discard vs fstrim:** Prefer periodic `(fstrim)` (e.g. a weekly systemd timer) over the continuous `(discard)` mount option, which can add latency to every delete.

fsync / O_DIRECT / wal_sync_method / write cache

- **Never disable `(fsync)`** (Percona's Charly Batista: "just never change it") — it guarantees crash safety by flushing WAL to durable media.
- **wal_sync_method:** default on Linux is `(fdatasync)` (buffered I/O + fsync). `(open_datasync)`/`(open_sync)` use O_DIRECT|O_DSYNC for WAL. Leave the platform default unless `(pg_test_fsync)` shows a clear win; `(fdatasync)` is robust and avoids the WAL-buffer-fill commit regression seen with `(open_datasync)`.
- **Write cache / BBU:** Volatile disk/controller caches must be flushed by fsync/barriers. With a battery-backed or flash-backed cache the write is non-volatile, so you may safely disable barriers (`(nobARRIER)`) for a large write-performance gain. Validate BBU health regularly.
- **fsync failure ("fsyncgate"):** The fix (authored by Craig Ringer, committed by Thomas Munro in November 2018) shipped in the Feb 14 2019 minor releases (11.2, 10.7, 9.6.12, 9.5.16, 9.4.21). Per the commit message, PostgreSQL will "violently prevent any future checkpoint attempts by panicking on the first fsync() failure," controlled by the new GUC `(data_sync_retry)` (default `(off)`, i.e. PANIC-and-recover). Ensure HA/failover is in place so this is survivable. ([PostgreSQL](#))
- **Alignment/stripe:** Align partitions to the device's erase-block/stripe; match RAID stripe to 8 kB page multiples; place WAL on a separate low-latency device where possible (WAL writes are sequential and benefit from dedicated bandwidth).

4. Kernel Scheduler and CPU

CPU frequency governor

- **Where:** `(/sys/devices/system/cpu/cpu*/cpufreq/scaling_governor)`; via `(intel_pstate)`/`(amd_pstate)`, cpupower, or Tuned (`(throughput-performance)`/`(latency-performance)`).
- **Recommended:** `(performance)` (pin max frequency).
- **Why:** `(power-save)`/`(ondemand)`/`(schedutil)` add latency while the CPU ramps up from low P-states; for latency-sensitive DB commits and short transactions the ramp-up cost hurts. Also disable deep C-states for lowest latency (via `(intel_idle.max_cstate)`/`(processor.max_cstate)` or a Tuned profile). Microsoft's SQL-Server-on-Linux Tuned profile similarly sets the governor and energy-perf bias for throughput.

CFS/EEVDF scheduler tunables

- Newer kernels (6.6+) replace CFS with **EEVDF**. Relevant knobs: `(kernel.sched_migration_cost_ns)` (raise, e.g. 5000000, to reduce task migration between cores for cache locality on many-connection servers), `(kernel.sched_autogroup_enabled=0)` (disable on servers so DB processes aren't grouped/deprioritized). `(sched_latency_ns)`/`(sched_min_granularity_ns)` exist on CFS kernels; test before changing. These are second-order compared to memory/IO settings.

CPU affinity / pinning

- Generally leave PostgreSQL unpinned and let the scheduler balance; for extreme NUMA setups you may pin the postmaster to a node with local memory, but this trades flexibility for locality. Keep `(irqbalance)` on (or manually pin NIC/NVMe IRQs) for even interrupt distribution.

5. Network Parameters

- **net.core.somaxconn**: listen backlog. Default 128 (4096 on very recent kernels). Raise to 1024–4096+ for high connection churn; PostgreSQL passes this as the `(listen())` backlog, and overflow yields "Resource temporarily unavailable"/"Connection refused."
- **net.core.netdev_max_backlog**: packet ingress queue; raise (e.g. 16384) on 10 GbE+.
- **net.ipv4.tcp_max_syn_backlog**: half-open connection queue; raise (e.g. 4096–8192) for connection storms.
- **TCP buffers**: `(net.core.rmem_max)(wmem_max)` (e.g. 16 MB on 1–10 GbE) and `(net.ipv4.tcp_rmem)(tcp_wmem)` (e.g. `(4096 87380 16777216)`). Keep TCP autotuning `(tcp_moderate_rcvbuf=1)`. Larger buffers help high-BDP replication links; on LAN, defaults are usually fine.
- **net.ipv4.ip_local_port_range**: widen (e.g. `(1024 65535)`) for hosts making many outbound connections (poolers, replication).
- **net.ipv4.tcp_tw_reuse=1 / tcp_fin_timeout**: `(tcp_tw_reuse=1)` safely reuses TIME_WAIT sockets for outbound connections; lower `(tcp_fin_timeout)` (e.g. 30) frees sockets faster. **Never** set `(tcp_tw_recycle)` (removed in kernel 4.12; breaks NAT and virtualized environments).
- **TCP keepalives + PostgreSQL GUCs**: kernel defaults are `(tcp_keepalive_time=7200)` (2 h), `(tcp_keepalive_intvl=75)`, `(tcp_keepalive_probes=9)`. To detect dead replication/client connections faster, set PostgreSQL's `(tcp_keepalives_idle)(interval)(count)` (per-server; a value of 0 uses the OS default) or tune the kernel. PG14+ adds `(client_connection_check_interval)` (Linux-only) to cancel a query when the client vanishes mid-query, and `(tcp_user_timeout)` for retransmit-based detection. Keepalives only detect dead **idle** connections; an active long query notices a dead client only when it next tries to write to the socket (Cybertec's Laurenz Albe: keepalive "would be more to the point named 'detectdead'").

6. Process / Resource Limits

- **nofile (open files)**: each connection, table file, and socket uses FDs. Raise via `(limits.conf)` `(postgres soft/hard nofile 65536+)` or systemd `(LimitNOFILE)`. Alternatively cap PostgreSQL's own usage with `(max_files_per_process)`.
- **nproc**: raise for high `(max_connections)` + background/parallel workers.
- **memlock (LimitMEMLOCK)**: must allow locking enough memory for explicit huge pages (`(LimitMEMLOCK=infinity)` is common in the systemd unit); otherwise `(huge_pages=on)` fails, and (as MySQL/InnoDB shows) an app that expected huge pages may fall back to normal memory and OOM.
- **core file size**: set nonzero + `(kernel.core_pattern)` when you need core dumps for gdb debugging.
- **systemd unit**: set `(LimitNOFILE)`, `(LimitNPROC)`, `(LimitMEMLOCK)`, and `(OOMScoreAdjust=-1000)` in the postgresql `([Service])` section (use a drop-in override, then `(systemctl daemon-reload)`). Watch `(RemoveIPC)` in `(logind.conf)` — if `(on)`, systemd can remove POSIX shm/semaphores when the postgres user "logs out," causing "could not remove shared memory segment" errors and breaking parallel query; keep it `(off)` for DB hosts.
- **cgroup v2 (MemoryMax/MemoryHigh)**: `(MemoryMax)` is a hard ceiling that triggers cgroup OOM (immediate SIGKILL, no ENOMEM) — size it ~20–30% above peak to account for page cache; use `(MemoryHigh)` ~10–20% below max as a throttle/reclaim point. `(MemorySwapMax=0)` to forbid swap. Monitor `(memory.pressure)` (PSI: watch `(some avg10)`). This is the container/Kubernetes analogue of overcommit tuning; note that inside a nested cgroup the child may report `(max)` even when constrained by a parent, so read effective limits carefully.
- **kernel.shmmax/shmall/shmmni (System V)**: Since 9.3, PostgreSQL uses POSIX `(mmap)` shared memory and needs only ~48 bytes of SysV shm, so defaults suffice unless `(shared_memory_type=sysv)`. Still relevant for huge-page sizing and some legacy setups.
- **kernel.sem (SEMMSL SEMMNS SEMOPM SEMMNI) / semaphores**: PostgreSQL needs one semaphore per allowed connection + autovacuum worker + background worker + WAL sender, in sets of 16 (+1 "magic" per set). On Linux, POSIX semaphores are used (no fixed kernel limit), but if using SysV: `(SEMMNI ≥ ceil((max_connections + autovacuum_max_workers + max_worker_processes + 5)/16))`, `(SEMMSL ≥ 17)`, `(SEMMNS)` sized accordingly. A typical value `(kernel.sem = 250 32000 100 128)` covers most deployments, including hundreds of concurrent users. Symptom of exhaustion: the confusingly worded "No space left on device" from `(semget)`.

7. Time / Clock

- **clocksource (tsc vs hpet vs acpi_pm)**: `(/sys/devices/system/clocksource/clocksource0/current_clocksource)`. Prefer `(tsc)` when stable — it's read from a CPU register and is far faster than HPET (memory-mapped) or `acpi_pm`. The PostgreSQL 18 `(pg_test_timing)`

docs show TSC on an Intel i7-860 at "Per loop time including overhead: 35.96 ns," versus "722.92 ns" after switching to `acpi_pm`.

`PostgreSQL` `PostgreSQL`

- **Why (internals):** `EXPLAIN ANALYZE` and `track_io_timing` call `gettimeofday`/`clock_gettime` per node/row. A slow clocksource inflates timed operations dramatically — the docs note "the timing overhead per row is 68 ns... making the fully timed count statement take almost 70% longer," and on `acpi_pm` the sample `EXPLAIN ANALYZE` rose to 115.9 ms. Verify with `pg_test_timing`; if `tsc` isn't the default it may have been disabled for a reason (unstable TSC on old CPUs or aggressive power-saving). `PostgreSQL`
- Persist via GRUB `clocksource=tsc tsc=reliable` where appropriate; on RHEL real-time, forcing TSC and simultaneous core-frequency transitions avoids the AMD64 `gettimeofday` skew when `cpufreq` + TSC interact.

8. Other Relevant

- **kernel.core_pattern:** set a writable path or pipe for core dumps when debugging crashes with `gdb`; ensure `ulimit -c` (or `systemd` `LimitCORE`) allows cores.
- **kernel.yama.ptrace_scope:** default `1` blocks attaching `gdb`/`strace` to non-child processes. Set to `0` temporarily to attach to running backends for debugging (security trade-off; revert after).
- **kernel.perf_event_paranoid:** default `2`+ restricts unprivileged profiling. Set `-1` (or `1`) to allow `perf` profiling of backends / flame graphs; combine with `kernel.kptr_restrict=0` for kernel symbols. Security-sensitive — use on non-prod or transiently. (From kernel 5.9, `perf` can be granted `CAP_PERFMON` instead of relaxing the `sysctl` globally.)
- **kernel.panic / kernel.panic_on_oops:** for HA/fencing, set `kernel.panic=<seconds>` (e.g. 10) to auto-reboot after panic and `panic_on_oops=1` so a failed node reboots and lets the cluster fail over cleanly (STONITH-style behavior).
- **Entropy / random for SSL:** modern kernels (`getrandom`, `RDRAND`) rarely starve; on old kernels/VMs consider `rng-tools`/`haveged` to avoid TLS handshake stalls.
- **PostgreSQL 18 AIO + io_uring:** `io_method` defaults to `worker` (with `io_workers=3`); `io_uring` needs a `--with-liburing` build, Linux kernel ≥ 5.1 , and the `sysctl` `kernel.io_uring_disabled=0`. That `sysctl` exists from kernel 6.6 (0 = all processes may create `io_uring` instances; 1 = restrict to privileged / `io_uring_group`; 2 = disable for all). `worker` is the safer default because `io_uring` has a significant kernel security-exposure history (Google reported `io_uring` was involved in a majority of Linux kernel exploits it found in 2022; credativ notes `io_uring` "bypasses traditional syscall audit paths"). Related PG18 defaults (raised to reflect modern hardware per the PG18 release notes): `effective_io_concurrency=16` (was 1 in PG17; range 0-1000, 0 disables async requests), `maintenance_io_concurrency=16`, `io_combine_limit=128kB`, `io_max_combine_limit=128kB`, `io_max_concurrency=-1` (auto, capped at 64). AIO currently accelerates **reads only** (sequential scans, bitmap heap scans, `VACUUM`/`ANALYZE`), not writes or WAL. Max read-ahead $\approx$ `effective_io_concurrency \times io_combine_limit`. (Phoronix + 5)

Recommendations

Stage 1 — Baseline for any dedicated PostgreSQL host (do first):

1. `vm.swappiness=1`, `vm.overcommit_memory=2` (+ size `overcommit_ratio`/`overcommit_kbytes`), `postmaster` `OOMScoreAdjust=-1000` with `PG_OOM_ADJUST_FILE`/`VALUE=0`.
2. Disable THP (`transparent_hugepage=never` in GRUB or a `systemd` oneshot ordered before `postgres`).
3. `vm.zone_reclaim_mode=0`; on NUMA hardware also `kernel.numa_balancing=0`.
4. I/O scheduler `none` for NVMe (udev rule); `noatime` mount; periodic `fstrim`.
5. CPU governor `performance`; disable deep C-states.
6. `LimitNOFILE`, `LimitNPROC`, `LimitMEMLOCK=infinity` in the `systemd` unit.

Stage 2 — Size to workload: 7. Enable explicit huge pages sized from `shared_memory_size_in_huge_pages`, set `huge_pages=on` once stable; verify with `SHOW huge_pages_status`. 8. Set absolute dirty limits (`vm.dirty_background_bytes`/`vm.dirty_bytes`) tuned to storage bandwidth; verify checkpoint smoothness via `pg_stat_bgwriter`/`pg_stat_io` (PG16+). 9. Confirm `tsc` clocksource; run `pg_test_timing`. 10. Tune `net.core.somaxconn`, backlogs, and TCP keepalives (+ `client_connection_check_interval`) for connection count / replication topology; prefer a connection pooler over very high `max_connections`.

Stage 3 — Advanced / version-specific: 11. On PG18, test `io_method=worker` (default) vs `io_uring` (set `kernel.io_uring_disabled=0`, ensure a `liburing` build); tune `effective_io_concurrency`/`io_workers` on high-latency cloud storage (2-3 $\times$ read throughput gains have been reported on cloud EBS-class volumes). 12. In containers/K8s, set cgroup `MemoryMax`/`MemoryHigh` (not `overcommit`) and `MemorySwapMax=0`. `pganalyze`

Benchmarks/thresholds that change these:

- If `(pg_stat_io)/(pg_stat_bgwriter)` shows rising backend writes or checkpoint I/O spikes → tighten dirty limits and raise `(max_wal_size)/(checkpoint_completion_target)`.
- If `(perf)/flame` graphs show memory compaction (`(do_huge_pmd_anonymous_page)`, high system CPU) → THP is still on somewhere.
- If p99 latency is high on NVMe under mixed load → test `(mq-deadline)` and measure tail latency (Q2D vs D2C via `(blktrace)/(btt)`).
- If OOM events appear in `(dmesg)` → re-size the memory budget and cgroup limits; confirm postmaster `(oom_score_adj)`.

Caveats

- Kernel tuning is hardware- and workload-specific; there are no universal "good" values for dirty limits, read-ahead, or TCP buffers — measure. Percona explicitly warns kernel tuning is "harder to be prescriptive" than DB tuning, and "improvement gains are likely to be minimal" beyond the high-impact knobs.
- Some recommendations conflict in the community (notably `(vm.swappiness)` 0 vs 1 vs 60, and whether to `(numactl --interleave)`); this report takes the mainstream predictable-latency position but flags the debate.
- The `(kernel.io_uring_disabled)` sysctl and the PG18/PG19 NUMA features are new/evolving; verify against your exact PostgreSQL minor version and kernel.
- Copy-pasting third-party sysctl bundles is dangerous (e.g. the widely-copied `(tcp_tw_recycle=1)` breaks NAT). Always test changes on staging, keep prior values for rollback, and manage via `(/etc/sysctl.d)`, udev rules, and systemd drop-ins for persistence and auditability.

Cheat-Sheet: Recommended Values for a Dedicated PostgreSQL Production Server

Parameter	Where	Default	Recommended (dedicated)	Notes
<code>vm.overcommit_memory</code>	sysctl	0	2	Prevents OOM-killing postmaster
<code>vm.overcommit_ratio</code>	sysctl	50	~80 (or use <code>overcommit_kbytes</code>)	Size <code>CommitLimit</code> to workload
<code>vm.swappiness</code>	sysctl	60	1	Not 0 (OOM risk)
<code>vm.dirty_background_bytes</code>	sysctl	0 (uses ratio 10)	~storage cache size (e.g. 64-256MB)	Absolute > percentage on big RAM
<code>vm.dirty_bytes</code>	sysctl	0 (uses ratio 20)	few× background (e.g. 256MB-1GB)	Keep > background
<code>vm.dirty_expire_centisecs</code>	sysctl	3000	500-3000	Lower for latency-sensitive
<code>vm.zone_reclaim_mode</code>	sysctl	auto (often 1)	0	Critical on NUMA
<code>kernel.numa_balancing</code>	sysctl	1	0	Single dedicated cluster
<code>vm.nr_hugepages</code>	sysctl/GRUB	0	= <code>shared_memory_size_in_huge_pages</code> + headroom	With <code>huge_pages=on</code>
THP enabled/defrag	/sys or GRUB	madvise/always	never (or madvise)	Disable before <code>huge_pages</code>
<code>vm.max_map_count</code>	sysctl	65530	262144 (large deployments)	Many partitions/extensions
<code>vm.min_free_kbytes</code>	sysctl	auto	1-2GB on big-RAM hosts	Avoid reclaim stalls
<code>oom_score_adj</code> (postmaster)	systemd/proc	0	-1000	Children reset to 0 via <code>PG_OOM_ADJUST_*</code>
I/O scheduler (NVMe)	/sys + udev	none	none	<code>mq-deadline</code> for SATA/HDD
<code>read_ahead_kb</code>	/sys / blockdev	128	128 (OLTP) / 2048-8192 (OLAP)	
Mount options	fstab	relatime	noatime (+nobARRIER only w/ BBU)	XFS or ext4
<code>fsync</code>	postgresql.conf	on	on (never off)	
<code>wal_sync_method</code>	postgresql.conf	<code>fdasync</code>	<code>fdasync</code> (default)	Test with <code>pg_test_fsync</code>

Parameter	Where	Default	Recommended (dedicated)	Notes
CPU governor	/sys cpufreq	powersave/schedutil	performance	Disable deep C-states
kernel.sched_autogroup_enabled	sysctl	1	0	Server workloads
kernel.sched_migration_cost_ns	sysctl	500000	~5000000	Reduce migration, high-conn
net.core.somaxconn	sysctl	128/4096	1024-4096+	Listen backlog
net.core.netdev_max_backlog	sysctl	1000	16384	10GbE+
net.ipv4.tcp_max_syn_backlog	sysctl	varies	4096-8192	
net.core.rmem_max/wmem_max	sysctl	~212KB	16MB (1-10GbE)	Keep autotuning on
net.ipv4.tcp_rmem/tcp_wmem	sysctl	4096 87380 6291456	4096 87380 16777216	
net.ipv4.ip_local_port_range	sysctl	32768 60999	1024 65535	Poolers/replication
net.ipv4.tcp_tw_reuse	sysctl	2	1	Never tcp_tw_recycle
tcp_keepalives_idle/intvl/count	postgresql.conf	0 (=OS: 7200/75/9)	e.g. 60/10/6	Detect dead replicas/clients
client_connection_check_interval	postgresql.conf	0	e.g. 10s	PG14+, Linux only
LimitNOFILE	systemd	1024	65536+	
LimitMEMLOCK	systemd	64K	infinity	For huge pages
kernel.sem	sysctl	varies	250 32000 100 128	Sized to connections+workers
kernel.shmmax/shmall	sysctl	large	default OK (POSIX shm since 9.3)	Only for sysv shm type
clocksource	/sys + GRUB	tsc (usually)	tsc	Verify with pg_test_timing
kernel.yama.ptrace_scope	sysctl	1	0 (transiently, for debugging)	Security trade-off
kernel.perf_event_paranoid	sysctl	2+	-1 or 1 (for profiling)	Non-prod/transient
kernel.panic / panic_on_oops	sysctl	0/varies	panic=10, panic_on_oops=1	HA fencing
kernel.io_uring_disabled	sysctl	0	0 (if using io_uring)	Kernel 6.6+; PG18 io_method
io_method (PG18)	postgresql.conf	worker	worker (test io_uring)	io_workers=3 default
effective_io_concurrency (PG18)	postgresql.conf	16	16-200 (SSD/NVMe, test)	Was 1 in PG17; range 0-1000
MemoryMax / MemoryHigh (cgroup)	systemd	infinity	Max ~peak+25%, High ~10-20% below	Containers/K8s; MemorySwapMax=0